TECHNICAL DESIGN DOCUMENT · 09 NOVEMBER 2026

GitHub URL: https://github.com/ikshith-techie/sahavas-township
Live app: Not deployed.

A calmer way to run township life.

Sahavas is an installable, responsive resident and service-desk app for maintenance, complaint follow-up and bank-statement reconciliation.

CORE BUILD · WORKING DEMO

Scope, deliberately small. Three seeded homes demonstrate sign-in, a maintenance bill and test payment, complaint triage and status, staff bank matching with explicit approval, and a committee snapshot. It is not a production federation platform or a full 2,400-flat deployment.

Prepared for: Company A · Pune | **Prototype:** Sahavas Township
All resident and transaction data in this prototype is synthetic and based on the problem statement.

1. Product and architecture

The hardest end-to-end risk in the brief is a mistaken bank allocation: a plausible but incorrect match can create an inaccurate flat ledger. The design therefore automates the *suggestion*, not the accounting decision. A staff member must explicitly post every confidently matched row; ambiguous rows cannot be posted through the normal action.

Layer	Choice	Why
Client	Responsive vanilla JavaScript, HTML and CSS; installable web app shell.	Small, inspectable build, fast mobile interaction, and no UI framework/runtime dependency.
Server	Node.js native HTTP server and JSON endpoints.	One process serves the UI and API; no external service is needed to run the demo.
Database	SQLite through Node's built-in `node:sqlite`; persistent local file or mounted host disk.	Transactions, constraints and durable demo data without a separate database service. Render blueprint attaches a persistent 1 GB disk.
Authentication	Mock resident OTP and staff password; random server-side session; HttpOnly, SameSite cookie.	Demonstrates role checks and session behavior without sending SMS or storing real contact details.

> **Assumption A1.** A supported reviewer runtime is Node.js 22.5 or later, where `node:sqlite` is available. The project was verified with Node.js 26.

> **Assumption A2.** This is a local/demo deployment. The fixed OTP and demo staff credential are not suitable for an internet-facing production deployment.

2. Data model

Table	Key fields and relationship	Purpose
`flats`	Flat ID (primary key), resident display name, area, synthetic UPI ID, sample overdue days.	Demo identity and billing/matching attributes.
`bills`	Bill ID; flat foreign key; period; amount; paid; due date; unique flat + period.	Monthly maintenance balance: amount minus recorded payments.
`payments`	Payment ID; flat and bill foreign keys; amount; unique reference; source; timestamp.	Receipt history for test payments and manually posted bank rows.
`complaints`	Complaint ID; flat foreign key; text; category; urgency; status; team/assignee; response due time; optional duplicate link; resolution time.	Track requests and calculate a resolution-time measure.
`bank_rows`	Row ID; import batch; original row JSON; match JSON; posted flag.	Keep each preview separate from the payment ledger until staff post.

Database writes use parameterized SQL. A test payment writes the payment and increments the bill's paid amount. A posted bank row repeats the balance check immediately before writing, preventing stale previews from overpaying. Duplicate imports do not silently create payments; a row can be posted once.

> **Assumption A3.** The submitted sample is the complete data set for the demo. Collection percentages, ageing and complaint counts are calculations over three sample flats, not estimates for all 2,400 homes.

3. Main flows

Resident sign-in and maintenance

1. Resident enters a flat ID and demo OTP; the server validates the flat and issues a role-bound session.
2. The home shows the current balance, open requests and recent activity. Maintenance shows area × ₹6.50, amounts received, balance and receipt history.
3. The test payment settles the outstanding balance, creates a transaction reference and refreshes the ledger. It moves no money and does not call a payment provider.

CSV reconciliation

1. Help desk staff uploads or pastes CSV; parser accepts quoted fields and requires amount plus reference/description.
2. For each row, exact outstanding amount is required, together with a flat ID in the reference or an exact synthetic UPI ID. Rows are labeled “Ready to post”, “Needs review” or “Unmatched”.
3. Preview alone never changes a bill. Staff explicitly posts a ready row; the server rechecks current balance, writes payment and receipt, then marks the row posted.
4. Unclear references, conflicting identifiers, wrong amounts, missing columns or invalid values surface as errors/queue states; they are not assigned to a guessed account.

> **Assumption A4.** This first pass treats rupee amounts as positive whole numbers, uses the latest bill for each flat, and allows either a flat reference or exact UPI ID as the independent identifier. Multi-bill allocation, refunds, bank formats beyond the documented CSV headers, and settlement-date rules need federation sign-off.

Complaints and triage

Residents submit free text. A local deterministic classifier suggests category and urgency, maps the category to a team and sets a response deadline. “Fire”, “gas”, “lift stuck” and “shock” are hard-coded critical safety signals and route to the duty manager regardless of any ordinary category rule. Similar open requests with the same category are linked by a small word-overlap check; the new report remains open. Help desk staff can change category and status; changes are persisted and reflected on resident and committee screens.

> **Automation limit:** This is transparent rule-based triage, not a trained or generative AI model. It runs locally, requires no API key and makes no claim of model confidence. “General” is a safe fallback; the help desk owns the correction path. It should be evaluated on consented, reviewed examples before broader use.

Committee snapshot

Staff-only view reports sample billed, collected and outstanding totals, percentage collected, ageing buckets and average elapsed time for resolved complaints. Unresolved requests show no invented resolution metric. The view is live from the same SQLite records, not a manually assembled spreadsheet export.

4. UI and design decisions

- **Resident home:** outstanding amount is the dominant figure; a single payment action and clear paid state reduce the time to the most common task. Open complaints and recent receipts are directly below.
- **Maintenance:** separates billed, paid and balance, and explains the ₹/sq-ft calculation. It does not invent an interest amount from unspecified bye-laws.
- **Complaint flow:** one short form, optional explicit category suggestion and visible safety escalation; status, assigned team and response timer stay on the request card.
- **Bank matching:** input and safeguards are side by side; each row displays its reason and posting state. “Nothing posts automatically” is kept visible.
- **Committee:** four summary measures first, then ageing, service health and account-level follow-up. Every number is derived from seeded/sample records.

Visual language uses warm white, deep green, restrained sage and amber; rounded cards, short labels and high-contrast key actions. Layout collapses to a mobile sidebar/menu and single-column cards. PWA shell assets cache for a quick revisit; API writes and private account data are not cached offline. There are explicit loading, empty, retry/error, success-toast and paid states.

> **Assumption A5.** English is the prototype language. Marathi/Hindi translations, translated content review and locale-specific message templates are not implemented yet.

5. Security, validation and tests

- No third-party payment, SMS or AI API calls; no secrets or real personal data are required.
- Session cookie is HttpOnly and SameSite=Strict, expires after eight hours, and is Secure when `NODE_ENV=production`. Resident and staff routes are role-gated.
- Server validates descriptions, enum updates, CSV size/schema/amounts and JSON bodies. SQL values are parameterized. Static responses set a restrictive same-origin content policy.
- Automated tests cover critical complaint safety rules, CSV quote parsing, exact-match gating, anonymous and cross-role access, test payment/receipt, no-auto-post preview, manual bank posting and double-post rejection.

Before production: replace mock authentication with OTP delivery and secure password management, add CSRF/origin policy, audit logs, backups, rate limits, privacy/legal review, multi-process session storage, and independent security testing.

6. Scope cuts and next steps

Excluded from this core: visitor approvals and photo retention, amenity bookings, notices administration, family/tenant relationship management, real gateways/UPI, real bank feeds and Tally export, file/photo complaint

uploads, rating on closure, multilingual UI, push/SMS, offline guard sync, and full-cohort/report exports. These exclusions avoid implying that mock interfaces safely process real money, identity or gate operations.

1. Validate bye-law billing, late-fee rules and CSV layouts with the federation/accounts team; add reviewed interest and reconciliation cases.
2. Connect an audited payment provider in test mode, implement idempotent webhooks and receipt reconciliation.
3. Replace demo sign-in, add resident/tenant/family relations and notification preferences, and complete English/Marathi/Hindi review.
4. Build complaint photo handling and retention, visitor/guard flows with encrypted offline queue, then amenity scheduling.
5. Run usability and accessibility tests with residents, guards and staff; load-test with all 2,400 flats and operational histories; deploy with monitored backups and an approved privacy/security assessment.

> **Assumption A6.** Render was selected for the hosted demo. An owner must still sign in, authorize the GitHub repository and confirm the paid service/disk in Render; no live app URL is claimed before that step completes. No SMS provider, payment provider, bye-law schedule or approved late-fee policy was supplied.